

Indian Liver Patient Dataset

This Dataset consists of Patients who has liver diseases and their medical history. It contains 416 liver patient records and 167 non liver patient records. Let's Analyse and preprocess the dataset based on their test samples.

Attribute Information

- Age
- Gender
- Total_Bilirubin
- Direct_Bilirubin
- Alkaline_Phosphotase
- Alamine_Aminotransferase
- Aspartate_Aminotransferase
- Total_Protiens
- Albumin
- Albumin_and_Globulin_Ratio
- target - Target Variable

```
In [1]: # Importing Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings("ignore")
```

```
In [2]: # Load the Dataset
data = pd.read_csv("C:\\Users\\rache\\OneDrive\\Rachel Study Materials\\Project Dataset\\Indian_Liver_Patient_I
```

```
In [3]: # Shape of the data
data.shape
```

```
Out[3]: (583, 11)
```

```
In [4]: # First Five rows of the dataset
data.head()
```

```
Out[4]:
```

	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_Amin
0	65	0	0.7	0.1	187		16
1	62	1	10.9	5.5	699		64
2	62	1	7.3	4.1	490		60
3	58	1	1.0	0.4	182		14
4	72	1	3.9	2.0	195		27

```
In [5]: # datatypes
data.dtypes
```

```
Out[5]:
```

Age	int64
Gender	int64
Total_Bilirubin	float64
Direct_Bilirubin	float64
Alkaline_Phosphotase	int64
Alamine_Aminotransferase	int64
Aspartate_Aminotransferase	int64
Total_Protiens	float64
Albumin	float64
Albumin_and_Globulin_Ratio	float64
target	int64
dtype:	object

- All Attributes are Numerical contains Integer and float values.

```
In [6]: # Information of the Dataset
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 583 entries, 0 to 582
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                    583 non-null    int64
1   Gender                                583 non-null    int64
2   Total_Bilirubin                       583 non-null    float64
3   Direct_Bilirubin                      583 non-null    float64
4   Alkaline_Phosphotase                  583 non-null    int64
5   Alamine_Aminotransferase              583 non-null    int64
6   Aspartate_Aminotransferase            583 non-null    int64
7   Total_Protiens                       583 non-null    float64
8   Albumin                              583 non-null    float64
9   Albumin_and_Globulin_Ratio            583 non-null    float64
10  target                                583 non-null    int64
dtypes: float64(5), int64(6)
memory usage: 50.2 KB
```

```
In [7]: # Summary of the dataset
data.describe()
```

```
Out[7]:
```

	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_Aminotransferase
count	583.000000	583.000000	583.000000	583.000000	583.000000	583.000000	583.000000
mean	44.746141	0.756432	3.298799	1.486106	290.576329	80.713551	80.713551
std	16.189833	0.429603	6.209522	2.808498	242.937989	182.620356	182.620356
min	4.000000	0.000000	0.400000	0.100000	63.000000	10.000000	10.000000
25%	33.000000	1.000000	0.800000	0.200000	175.500000	23.000000	23.000000
50%	45.000000	1.000000	1.000000	0.300000	208.000000	35.000000	35.000000
75%	58.000000	1.000000	2.600000	1.300000	298.000000	60.500000	60.500000
max	90.000000	1.000000	75.000000	19.700000	2110.000000	2000.000000	2000.000000

- From the summary of the dataset, we notice there are no missing values or null values.
- But the dataset is right skewed of the following Attributes **Total_Bilirubin, Direct_Bilirubin, Alkaline_Phosphotase, Alamine_Aminotransferase, Aspartate_Aminotransferase** which needs to be treated.

```
In [8]: # Null values
data.isnull().sum()
```

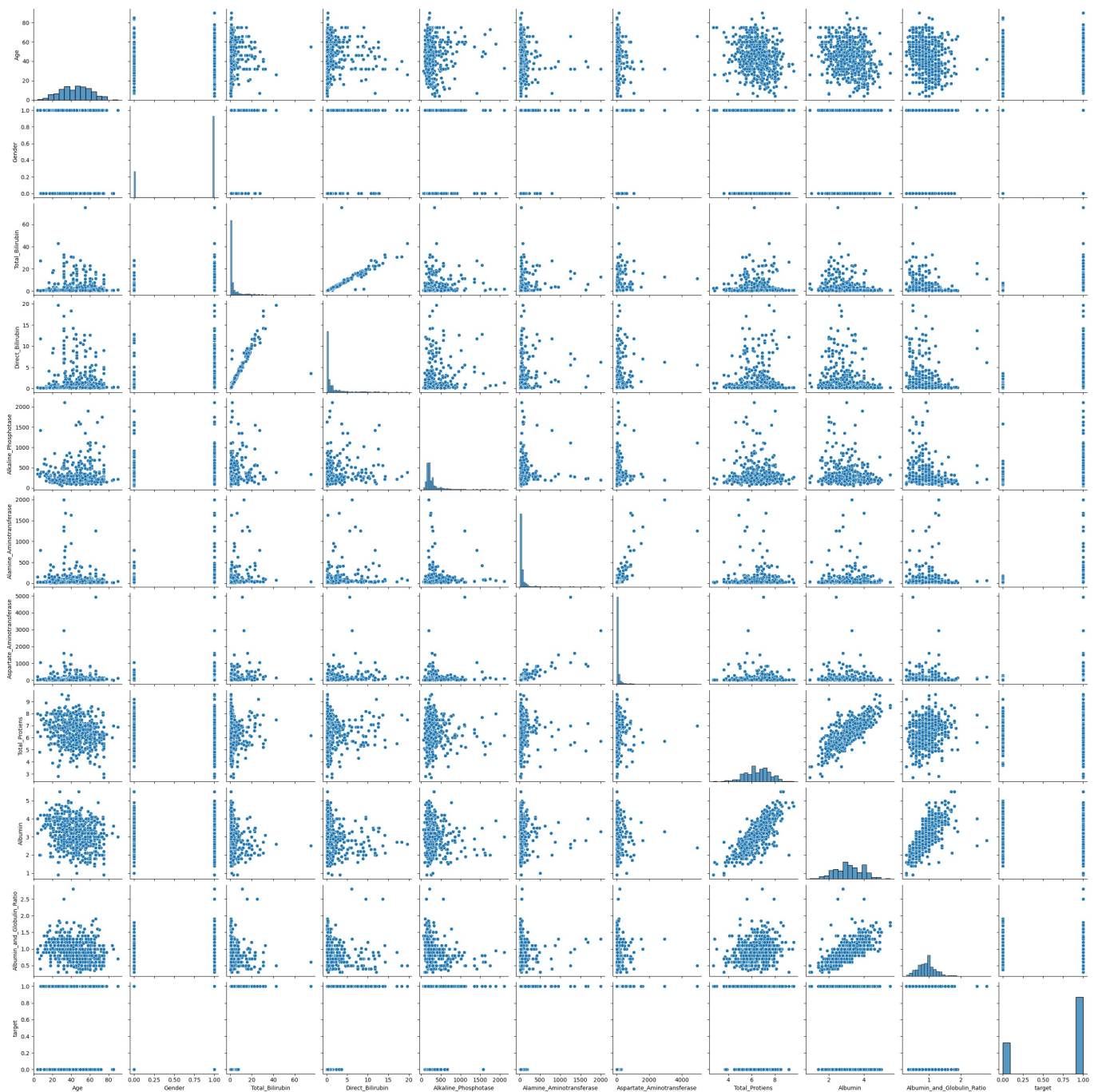
```
Out[8]: Age                                0
Gender                                0
Total_Bilirubin                       0
Direct_Bilirubin                      0
Alkaline_Phosphotase                  0
Alamine_Aminotransferase              0
Aspartate_Aminotransferase            0
Total_Protiens                       0
Albumin                              0
Albumin_and_Globulin_Ratio            0
target                                0
dtype: int64
```

- Checked the null values there are no null values, we'll move forward to Exploratory Data Analysis.

Exploratory Data Analysis (EDA)

```
In [9]: # Lets visualize the Whole dataset
sns.pairplot(data)
```

```
Out[9]: <seaborn.axisgrid.PairGrid at 0x1d75b536900>
```

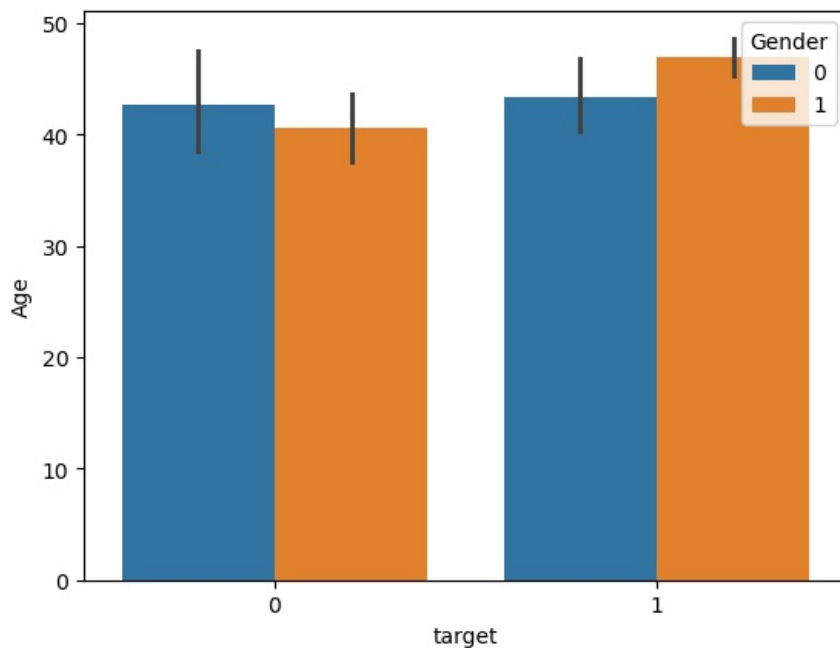


From this pairplot it clearly shows the dataset not evenly scattered or normally distributed and noticing some outliers

```
In [10]: # Barplot
print("Gender Difference between Liver Patients and Non Liver Patients with their Age")
sns.barplot(data = data, x = 'target', y = 'Age', hue = 'Gender')
```

Gender Difference between Liver Patients and Non Liver Patients with their Age

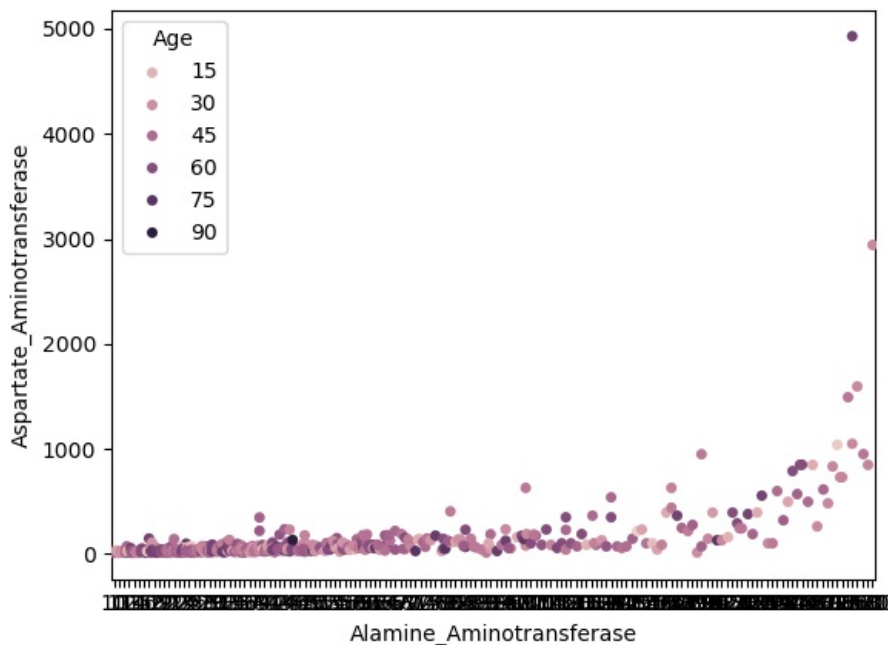
```
Out[10]: <Axes: xlabel='target', ylabel='Age'>
```



```
In [11]: # Swarmplot
print("Alamine and Aspartate Aminotransferse distribution based on the Age")
sns.swarmplot(data = data, x = 'Alamine_Aminotransferase', y = 'Aspartate_Aminotransferase', hue = 'Age')
```

Alamine and Aspartate Aminotransferse distribution based on the Age

```
Out[11]: <Axes: xlabel='Alamine_Aminotransferase', ylabel='Aspartate_Aminotransferase'>
```

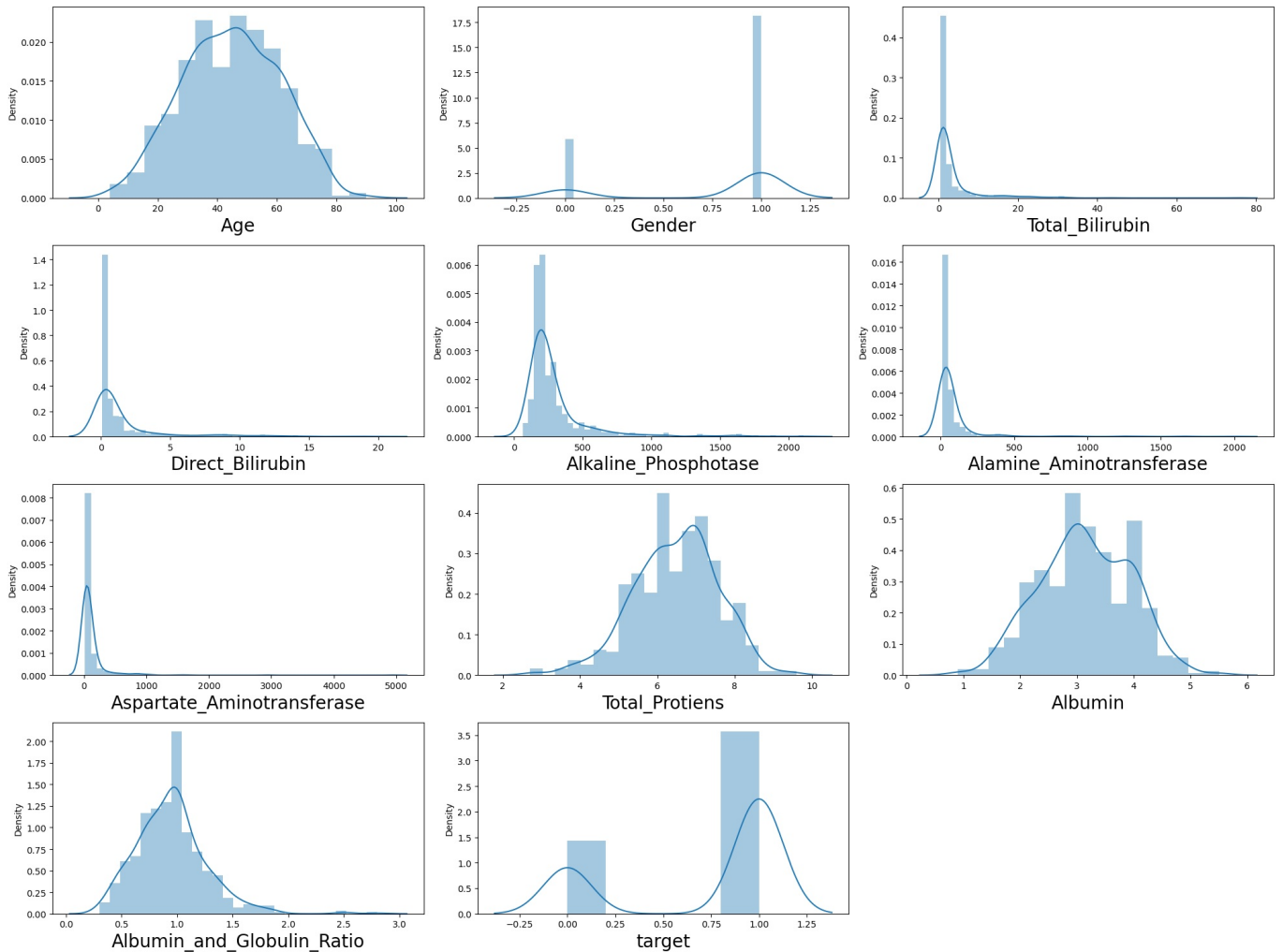


Data Preprocessing

- As already mentioned above we observed skewness in the dataset, let's transform the skewness and outliers using Log Transformation method and IQR Inter Quartile Range.

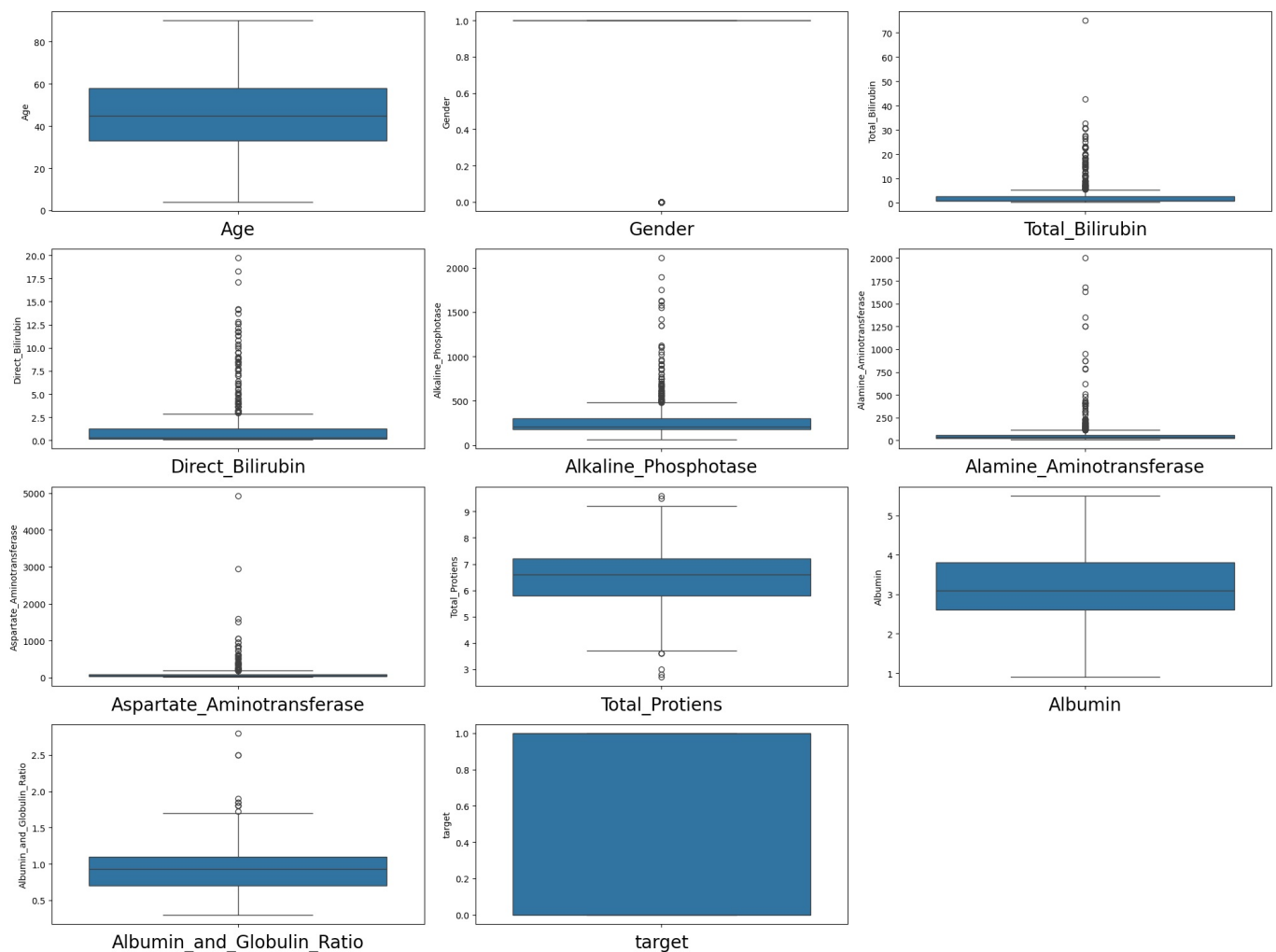
```
In [12]: # Checking the Skewness
plt.figure(figsize=(20,15))
plotnumber=1
for column in data:
    if plotnumber<=11:
        ax=plt.subplot(4,3,plotnumber)
        sns.distplot(data[column])
```

```
plt.xlabel(column,fontsize=20)
plotnumber +=1
plt.tight_layout()
```



The distplot Visualization clearly shows that there is Highly Right skewed in the "Total_Bilirubin, Direct Bilirubin, Alkaline_Phosphotase, Alamine_Aminotransferase, Aspartate_Aminotransferase".

```
In [13]: # Check the outliers
plt.figure(figsize=(20,15))
plotnumber=1
for column in data:
    if plotnumber<=11:
        ax=plt.subplot(4,3,plotnumber)
        sns.boxplot(data[column])
        plt.xlabel(column,fontsize=20)
        plotnumber +=1
plt.tight_layout()
```



The boxplot Visualization clearly shows that there is outliers present in the "Total_Bilirubin, Direct Bilirubin, Alkaline_Phosphotase, Alamine_Aminotransferase, Aspartate_Aminotransferase" which needs to be treated.

```
In [14]: data.skew()
```

```
Out[14]: Age                -0.029385
Gender              -1.197919
Total_Bilirubin      4.907474
Direct_Bilirubin     3.212403
Alkaline_Phosphotase 3.765106
Alamine_Aminotransferase 6.549192
Aspartate_Aminotransferase 10.546177
Total_Protiens       -0.285672
Albumin              -0.043685
Albumin_and_Globulin_Ratio 0.996780
target              -0.947140
dtype: float64
```

- Apart from Binary values, we'll understand the skewness from Continous Variables
- **Total_Bilirubin, Direct_Bilirubin, Alkaline_Phosphotase, Alamine_Aminotransferase, Aspartate_Aminotransferase** are highly Right skewed Attributes.
- **Albumin_and_Globulin_Ratio** is moderate skewed.
- There are no zero values let's move with treating outliers and skewness.

```
In [15]: # Treat the Outliers
# Let's find the IQR (Inter-Quartile Range) to identify the outliers
```

```
In [16]: #1st Quantile
q1 = data.quantile(0.25)
```

```
In [17]: #3rd Quantile
q3 = data.quantile(0.75)
```

```
In [18]: # IQR Range
IQR = q3 - q1
```

*Outliers Detection Formula**

Higher side: $Q3 + (1.5 * IQR)$

Lower Side: $Q1 - (1.5 * IQR)$

```
In [19]: print("1st Quantile:")
print(q1)
print(" ")
print("3rd Quantile:")
print(q3)
print(" ")
print("Inter Quartile Range:")
print(IQR)
```

```
1st Quantile:
Age                33.0
Gender             1.0
Total_Bilirubin    0.8
Direct_Bilirubin   0.2
Alkaline_Phosphatase 175.5
Alamine_Aminotransferase 23.0
Aspartate_Aminotransferase 25.0
Total_Protiens     5.8
Albumin            2.6
Albumin_and_Globulin_Ratio 0.7
target            0.0
Name: 0.25, dtype: float64
```

```
3rd Quantile:
Age                58.0
Gender             1.0
Total_Bilirubin    2.6
Direct_Bilirubin   1.3
Alkaline_Phosphatase 298.0
Alamine_Aminotransferase 60.5
Aspartate_Aminotransferase 87.0
Total_Protiens     7.2
Albumin            3.8
Albumin_and_Globulin_Ratio 1.1
target            1.0
Name: 0.75, dtype: float64
```

```
Inter Quartile Range:
Age                25.0
Gender             0.0
Total_Bilirubin    1.8
Direct_Bilirubin   1.1
Alkaline_Phosphatase 122.5
Alamine_Aminotransferase 37.5
Aspartate_Aminotransferase 62.0
Total_Protiens     1.4
Albumin            1.2
Albumin_and_Globulin_Ratio 0.4
target            1.0
dtype: float64
```

```
In [20]: #Validating Total Bilirubin Outlier
bil_pred_high = (q3.Total_Bilirubin + (1.5 * IQR.Total_Bilirubin))
bil_pred_high
```

```
Out[20]: np.float64(5.3000000000000001)
```

```
In [21]: # Index values of Total Bilirubin Higher Values
index=np.where(data['Total_Bilirubin']>bil_pred_high)
index
```

```
Out[21]: (array([ 1,  2, 22, 27, 37, 42, 49, 54, 55, 90, 91, 92, 93,
        97, 114, 115, 117, 118, 119, 120, 121, 126, 135, 154, 155, 160,
        161, 162, 166, 168, 169, 172, 177, 178, 179, 190, 191, 199, 259,
        260, 268, 272, 311, 332, 336, 370, 392, 401, 419, 420, 442, 447,
        450, 458, 478, 485, 503, 504, 505, 506, 508, 509, 519, 526, 531,
        535, 536, 544, 546, 547, 548, 553, 559, 560, 561, 562, 565, 568,
        569, 572, 574, 575, 576, 577]),)
```

```
In [22]: # Log Transformation Method
data['Total_Bilirubin'] = np.log1p(data['Total_Bilirubin'])
```

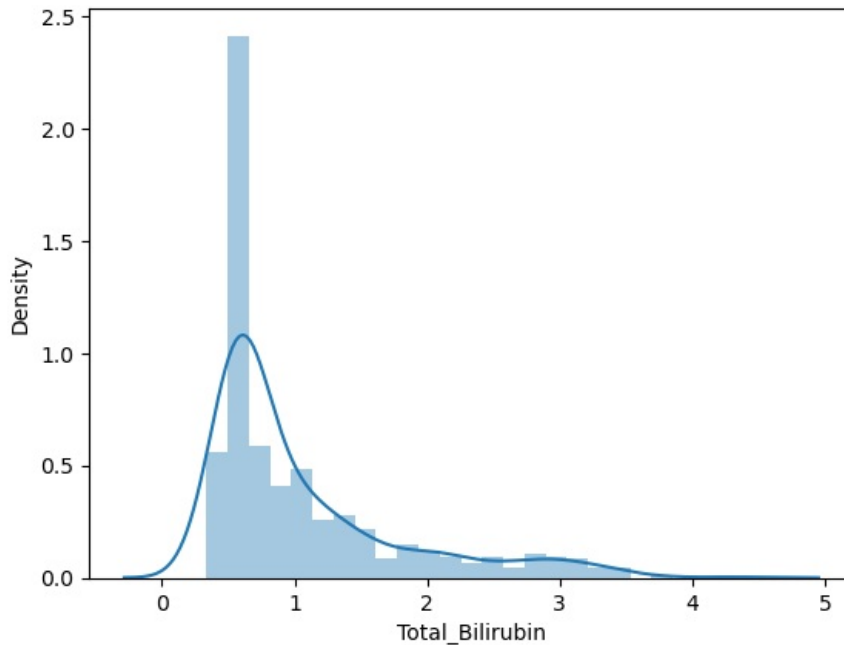
```
In [23]: data['Total_Bilirubin'] = np.where(data['Total_Bilirubin'] > bil_pred_high, bil_pred_high, data['Total_Bilirubin'])
```

```
In [24]: data.head()
```

Out[24]:	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_Amir
0	65	0	0.530628	0.1	187		16
1	62	1	2.476538	5.5	699		64
2	62	1	2.116256	4.1	490		60
3	58	1	0.693147	0.4	182		14
4	72	1	1.589235	2.0	195		27

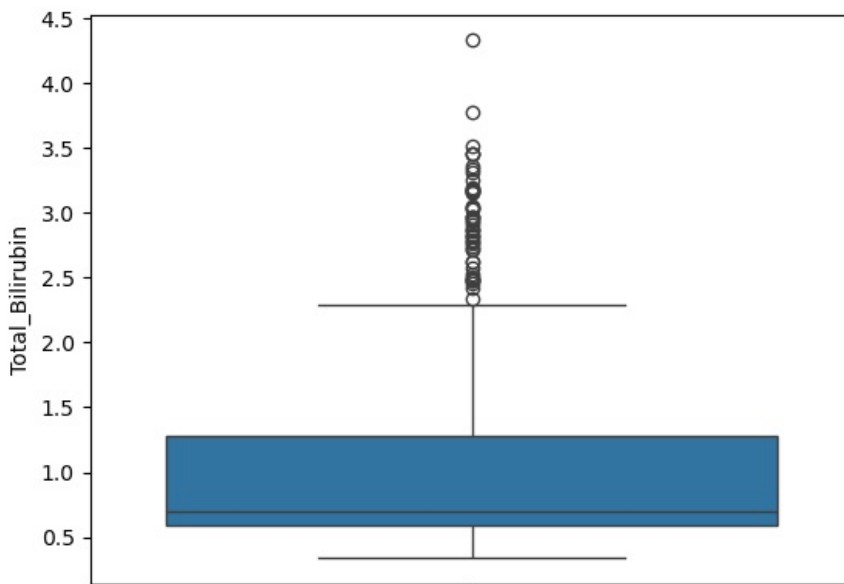
```
In [25]: sns.distplot(data['Total_Bilirubin'])
```

```
Out[25]: <Axes: xlabel='Total_Bilirubin', ylabel='Density'>
```



```
In [26]: sns.boxplot(data["Total_Bilirubin"])
```

```
Out[26]: <Axes: ylabel='Total_Bilirubin'>
```



- Using Log transformation We treated the outlier and skewness of Total_Bilirubin.

```
In [27]: # Treating Direct Bilirubin
DB_pred_high = (q3.Direct_Bilirubin + (1.5 * IQR.Direct_Bilirubin))
DB_pred_high
```

```
Out[27]: np.float64(2.95)
```

```
In [28]: # Index values of Direct Bilirubin have higher values
index=np.where(data['Direct_Bilirubin']>DB_pred_high)
index
```

```
Out[28]: (array([ 1,  2, 22, 27, 37, 42, 49, 54, 55, 91, 92, 97, 114,
        115, 117, 118, 119, 120, 121, 135, 155, 160, 161, 166, 168, 169,
        172, 177, 178, 179, 190, 191, 199, 246, 259, 260, 261, 268, 272,
        311, 332, 336, 370, 392, 401, 419, 420, 442, 447, 450, 458, 478,
        485, 503, 504, 505, 506, 508, 509, 519, 526, 531, 535, 536, 544,
        546, 547, 548, 553, 559, 560, 561, 562, 565, 568, 569, 572, 574,
        575, 576, 577]),)
```

```
In [29]: # Log Transformation Method
data['Direct_Bilirubin'] = np.log1p(data['Direct_Bilirubin'])
```

```
In [30]: # Transformed values to the columns
data['Direct_Bilirubin'] = np.where(data['Direct_Bilirubin'] > DB_pred_high, DB_pred_high, data['Direct_Bilirubin'])
```

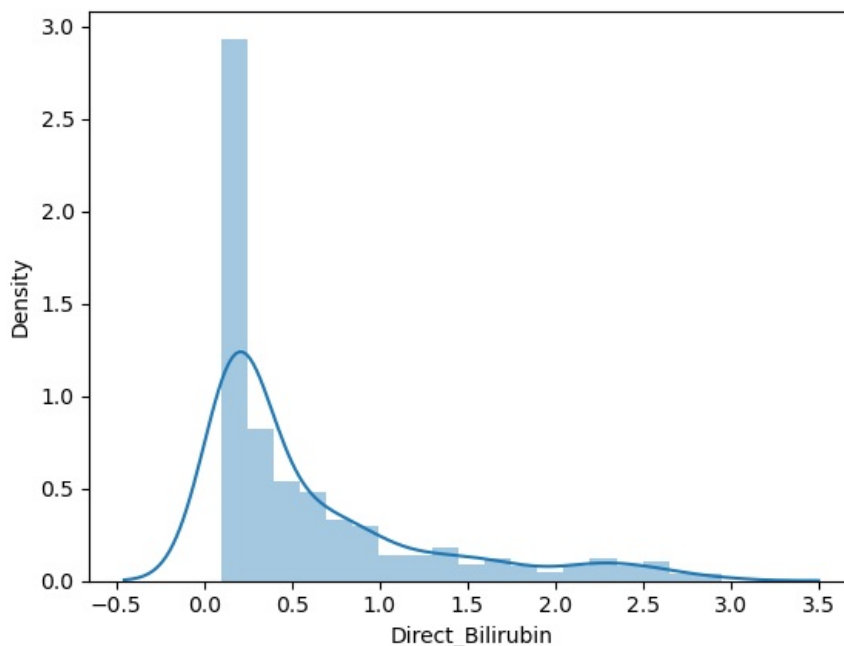
```
In [31]: data.head()
```

```
Out[31]:
```

	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_Amin
0	65	0	0.530628	0.095310	187		16
1	62	1	2.476538	1.871802	699		64
2	62	1	2.116256	1.629241	490		60
3	58	1	0.693147	0.336472	182		14
4	72	1	1.589235	1.098612	195		27

```
In [32]: sns.distplot(data['Direct_Bilirubin'])
```

```
Out[32]: <Axes: xlabel='Direct_Bilirubin', ylabel='Density'>
```



```
In [33]: # Treating Alkaline_Phosphotase
Alp_pred_high = (q3.Alkaline_Phosphotase + (1.5 * IQR.Alkaline_Phosphotase))
Alp_pred_high
```

```
Out[33]: np.float64(481.75)
```

```
In [34]: #Check the indexes which have higher values
index=np.where(data['Alkaline_Phosphotase']>Alp_pred_high)
index
```

```
Out[34]: (array([ 1,  2, 20, 21, 22, 35, 42, 76, 77, 79, 80, 88, 109,
        114, 115, 116, 126, 128, 130, 135, 160, 161, 162, 168, 169, 172,
        176, 177, 178, 186, 188, 195, 196, 199, 233, 249, 251, 255, 256,
        261, 285, 306, 309, 311, 341, 353, 370, 384, 393, 407, 408, 409,
        411, 415, 416, 417, 419, 420, 429, 447, 452, 459, 485, 492, 549,
        573, 574, 575, 578]),)
```

```
In [35]: data['Alkaline_Phosphotase'] = np.log1p(data['Alkaline_Phosphotase'])
```

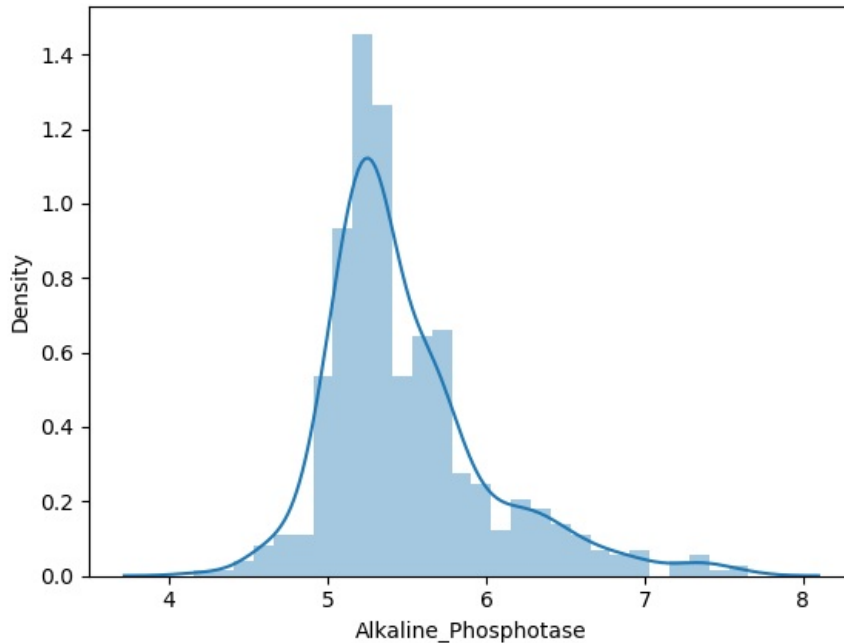
```
In [36]: data['Alkaline_Phosphotase'] = np.where(data['Alkaline_Phosphotase'] > Alp_pred_high, Alp_pred_high, data['Alka
```

```
In [37]: data.head()
```

Out[37]:	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_Amir
0	65	0	0.530628	0.095310	5.236442		16
1	62	1	2.476538	1.871802	6.551080		64
2	62	1	2.116256	1.629241	6.196444		60
3	58	1	0.693147	0.336472	5.209486		14
4	72	1	1.589235	1.098612	5.278115		27

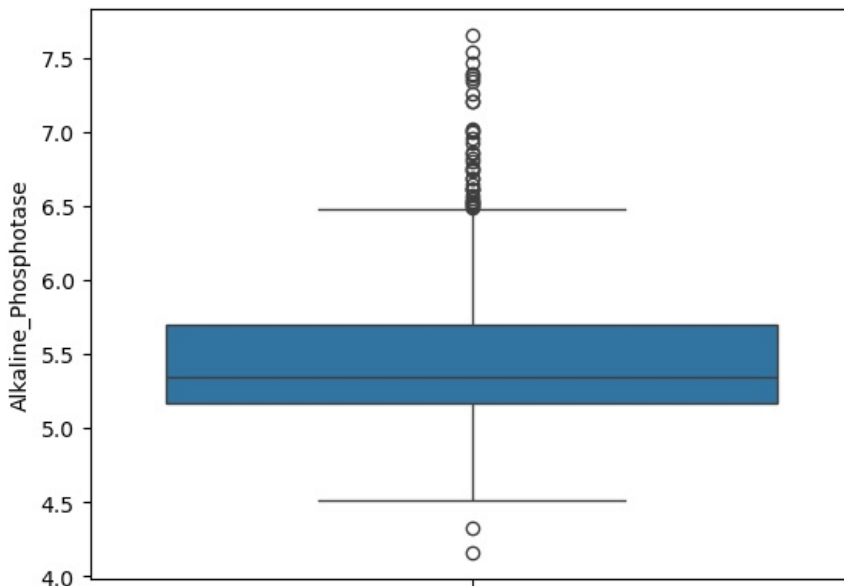
```
In [38]: sns.distplot(data['Alkaline_Phosphotase'])
```

```
Out[38]: <Axes: xlabel='Alkaline_Phosphotase', ylabel='Density'>
```



```
In [39]: sns.boxplot(data['Alkaline_Phosphotase'])
```

```
Out[39]: <Axes: ylabel='Alkaline_Phosphotase'>
```



```
In [40]: # Treating Alamine_Aminotransferase
Ala_pred_high = (q3.Alamine_Aminotransferase + (1.5 * IQR.Alamine_Aminotransferase))
Ala_pred_high
```

```
Out[40]: np.float64(116.75)
```

```
In [41]: #Check the indexes which have higher values
index=np.where(data['Alamine_Aminotransferase']>Ala_pred_high)
index
```

```
Out[41]: (array([ 16, 18, 19, 25, 26, 27, 39, 43, 47, 70, 76, 83, 89,
          90, 91, 92, 93, 94, 97, 99, 114, 117, 118, 119, 120, 121,
          126, 135, 160, 171, 190, 199, 207, 232, 235, 257, 261, 271, 272,
          289, 309, 318, 326, 336, 347, 393, 399, 402, 407, 408, 409, 410,
          447, 460, 475, 478, 479, 481, 485, 492, 494, 507, 508, 509, 519,
          533, 555, 558, 560, 561, 562, 569, 570]),)
```

```
In [42]: data['Alamine_Aminotransferase'] = np.log1p(data['Alamine_Aminotransferase'])
```

```
In [43]: data['Alamine_Aminotransferase'] = np.where(data['Alamine_Aminotransferase'] > Ala_pred_high, Ala_pred_high, da
```

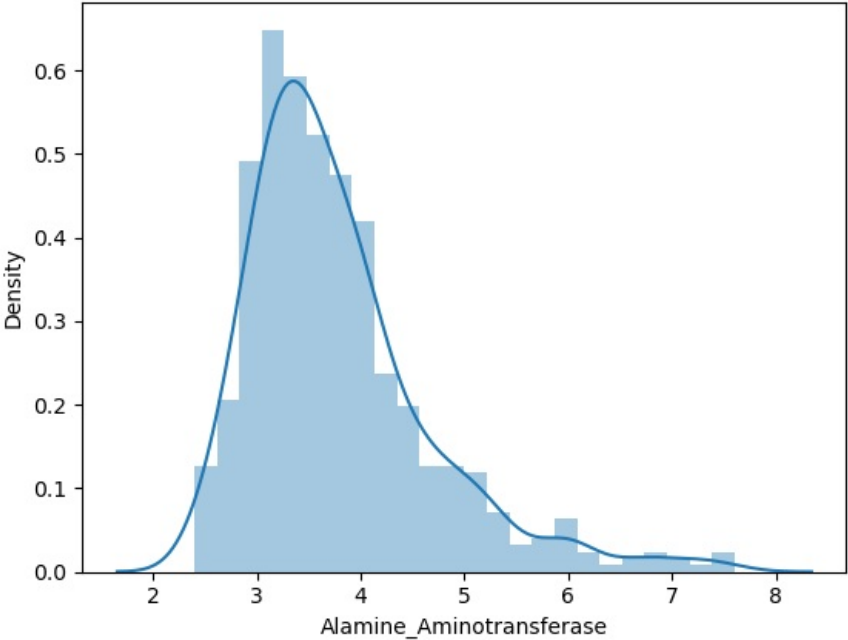
```
In [44]: data.head()
```

Out[44]:

	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_Amir
0	65	0	0.530628	0.095310	5.236442	2.833213	
1	62	1	2.476538	1.871802	6.551080	4.174387	
2	62	1	2.116256	1.629241	6.196444	4.110874	
3	58	1	0.693147	0.336472	5.209486	2.708050	
4	72	1	1.589235	1.098612	5.278115	3.332205	

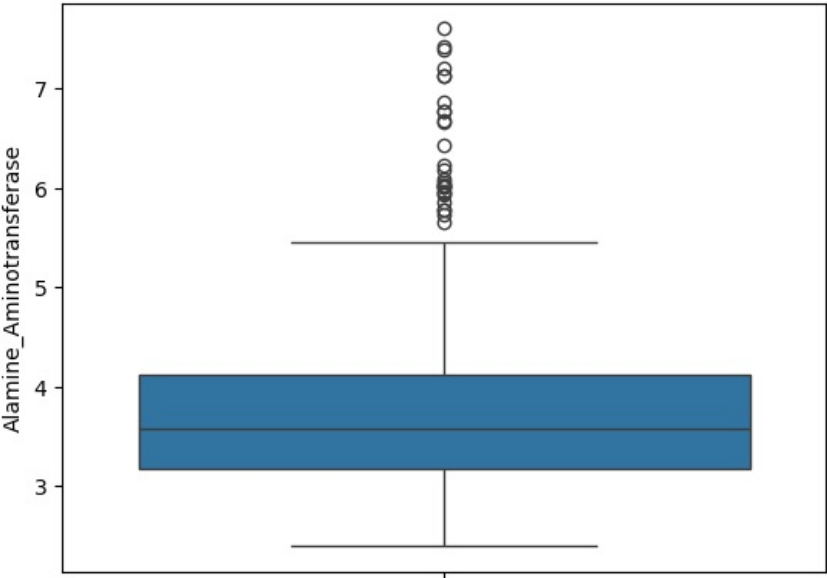
```
In [45]: sns.distplot(data["Alamine_Aminotransferase"])
```

```
Out[45]: <Axes: xlabel='Alamine_Aminotransferase', ylabel='Density'>
```



```
In [46]: sns.boxplot(data["Alamine_Aminotransferase"])
```

```
Out[46]: <Axes: ylabel='Alamine_Aminotransferase'>
```



```
In [47]: # Treating Aspartate Aminotransferase
Asp_pred_high = (q3.Aspartate_Aminotransferase + (1.5 * IQR.Aspartate_Aminotransferase))
Asp_pred_high
```

```
Out[47]: np.float64(180.0)
```

```
In [48]: #Check the indexes which have higher values
index=np.where(data['Aspartate_Aminotransferase']>Asp_pred_high)
index
```

```
Out[48]: (array([ 16,  18,  19,  25,  26,  27,  43,  47,  52,  70,  89,  90,  91,
        92,  93,  94,  96,  97,  99, 114, 117, 118, 119, 120, 135, 154,
       160, 161, 171, 187, 199, 207, 231, 235, 271, 272, 309, 318, 401,
       409, 410, 447, 478, 479, 481, 482, 485, 487, 494, 503, 505, 506,
       508, 509, 515, 519, 527, 531, 533, 536, 547, 548, 558, 560, 561,
       562]),)
```

```
In [49]: data['Aspartate_Aminotransferase'] = np.log1p(data['Aspartate_Aminotransferase'])
```

```
In [50]: data['Aspartate_Aminotransferase'] = np.where(data['Aspartate_Aminotransferase'] > Asp_pred_high, Asp_pred_high
```

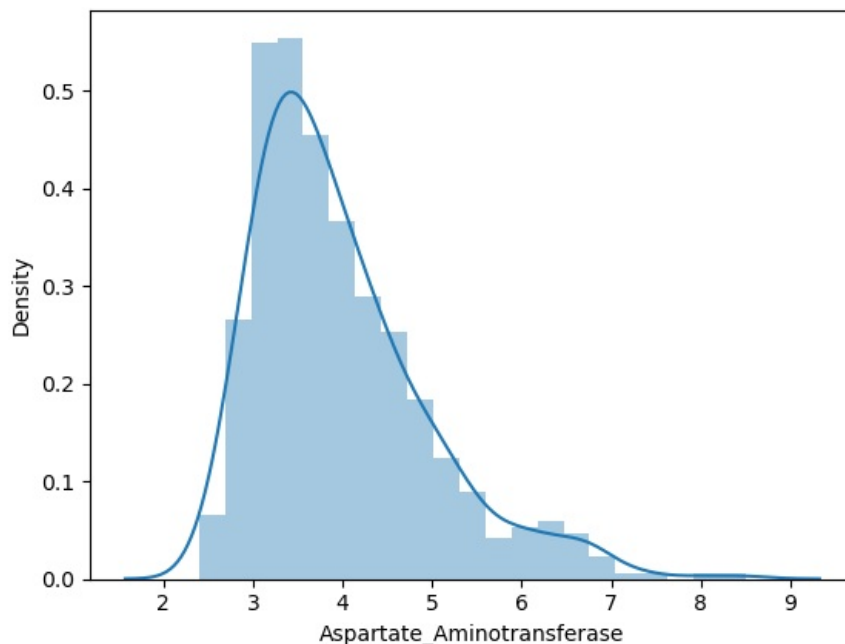
```
In [51]: data.head()
```

```
Out[51]:
```

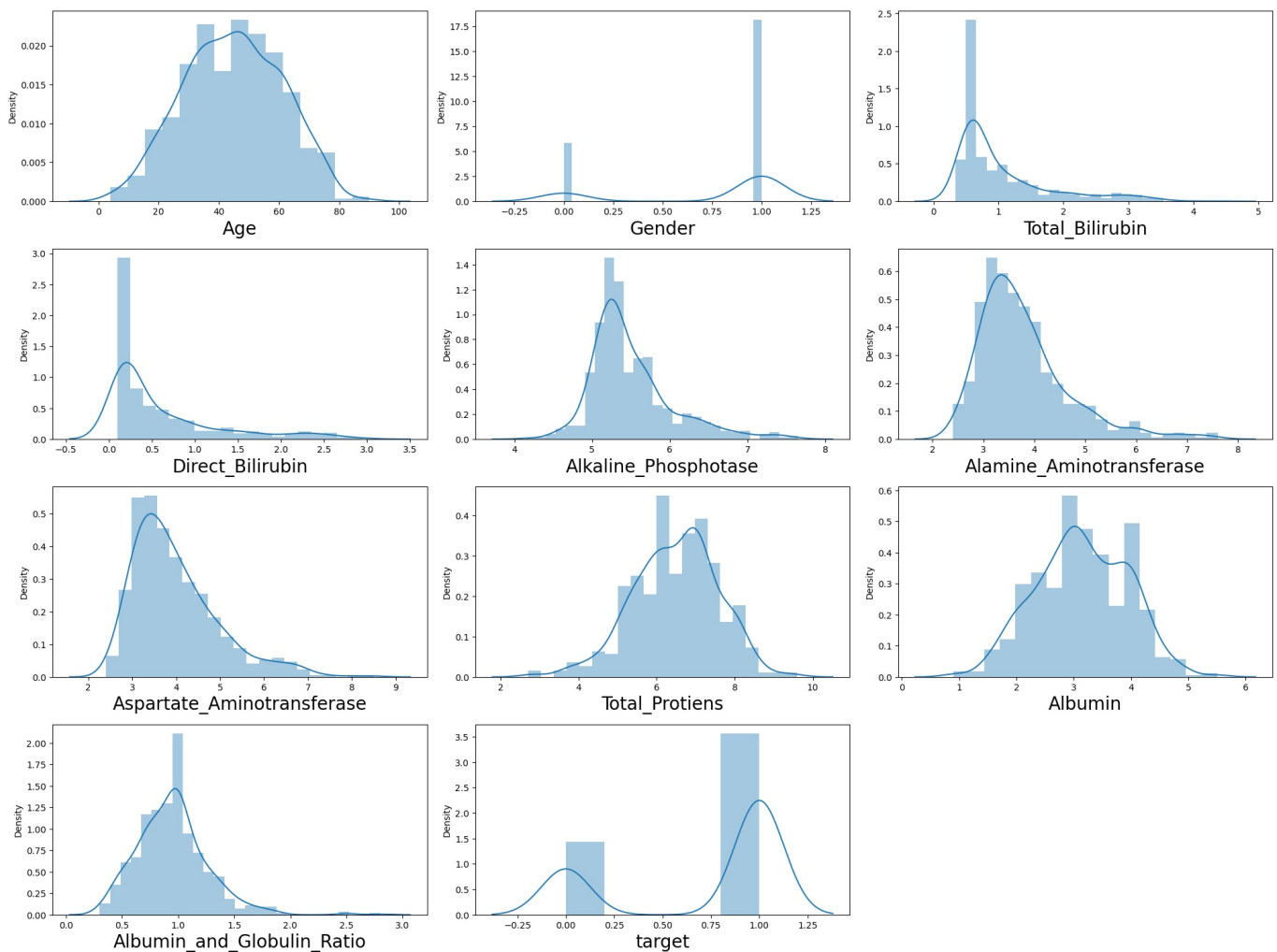
	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_Amin
0	65	0	0.530628	0.095310	5.236442	2.833213	
1	62	1	2.476538	1.871802	6.551080	4.174387	
2	62	1	2.116256	1.629241	6.196444	4.110874	
3	58	1	0.693147	0.336472	5.209486	2.708050	
4	72	1	1.589235	1.098612	5.278115	3.332205	

```
In [52]: sns.distplot(data['Aspartate_Aminotransferase'])
```

```
Out[52]: <Axes: xlabel='Aspartate_Aminotransferase', ylabel='Density'>
```

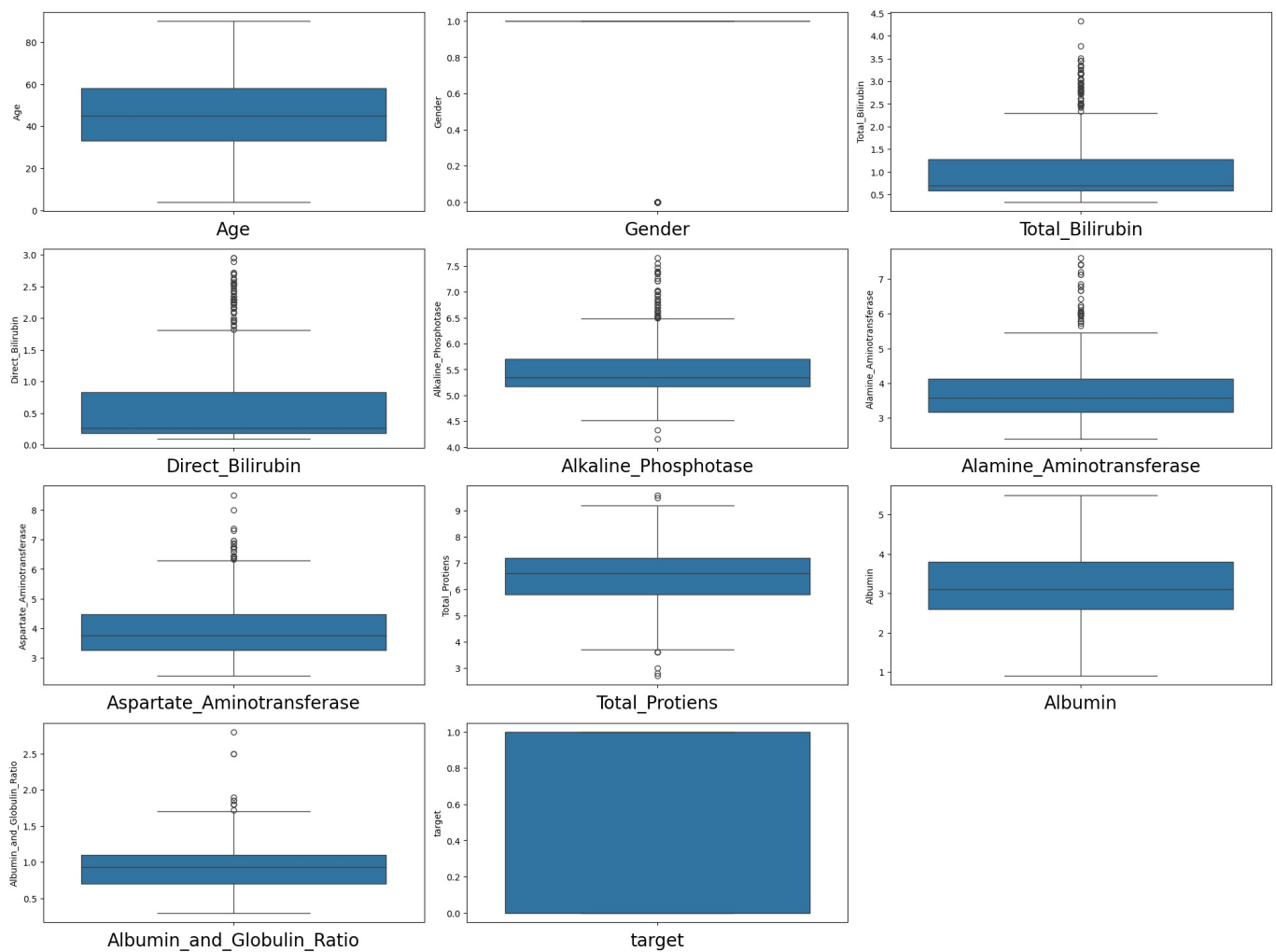


```
In [53]: # Checking the Skewness after treated the outliers
plt.figure(figsize=(20,15))
plotnumber=1
for column in data:
    if plotnumber<=11:
        ax=plt.subplot(4,3,plotnumber)
        sns.distplot(data[column])
        plt.xlabel(column,fontsize=20)
        plotnumber +=1
plt.tight_layout()
```



- Let's understand how this Skewness and outliers were treated.
- Since the dataset has 583 rows and there are many indexes rows which has higher values, so we can't able to drop the rows due to it leads to imbalance dataset that affects the model training.
- We can clearly see how well the skewness is treated without dropping any indexes.
- The Right Skewed attributes are **Normal Distributed**.

```
In [54]: plt.figure(figsize=(20,15))
plotnumber=1
for column in data:
    if plotnumber<=11:
        ax=plt.subplot(4,3,plotnumber)
        sns.boxplot(data[column])
        plt.xlabel(column,fontsize=20)
        plotnumber +=1
plt.tight_layout()
```

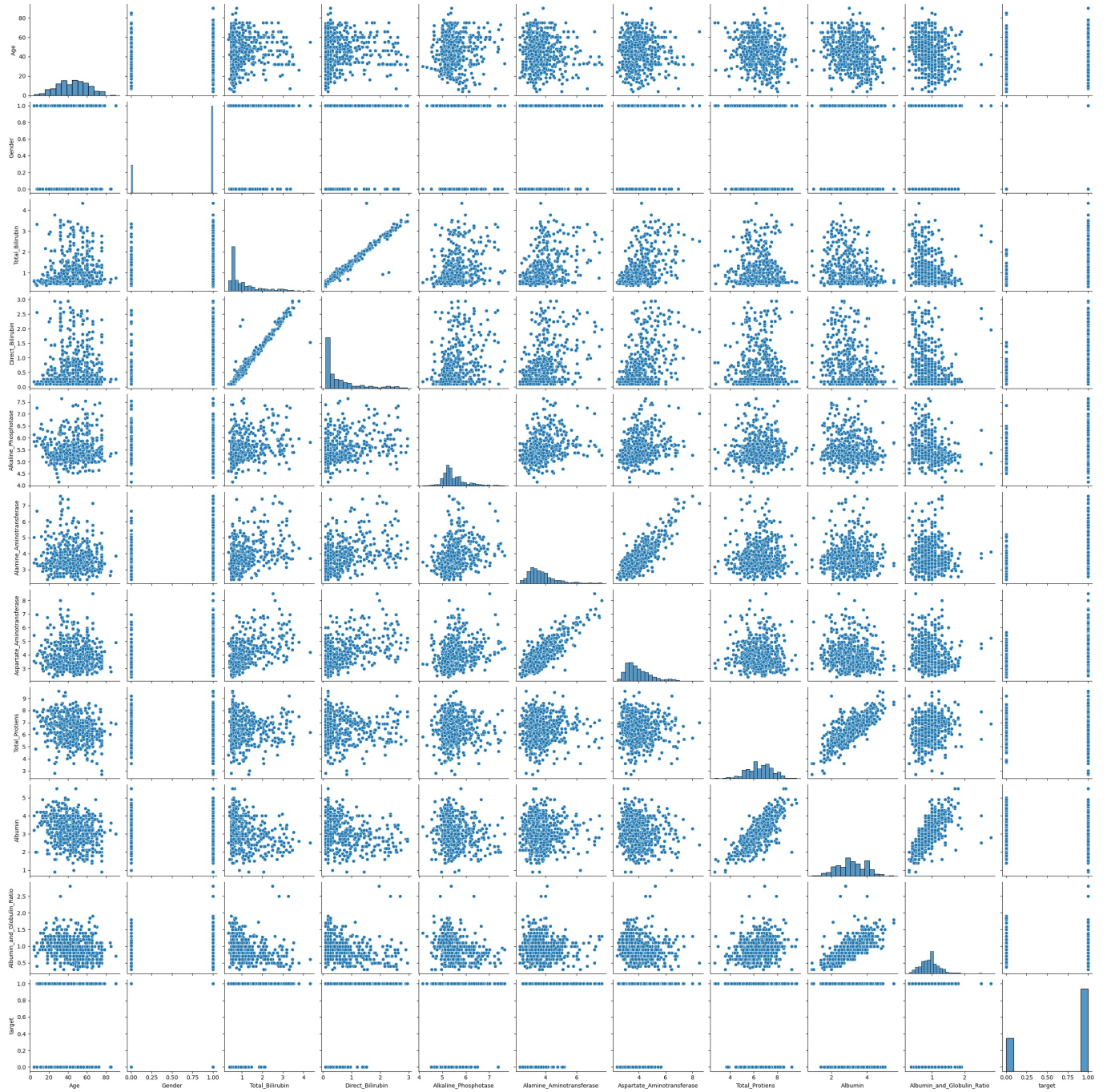


- All column outliers treated well without dropping any rows using Log Transformation method.
- Now All columns were normally distributed, Let's check the Multicollinearity problem to avoid the overfitting problem and to improve model stability.

Exploratory Data Analysis after Cleaned Dataset.

```
In [55]: sns.pairplot(data)
```

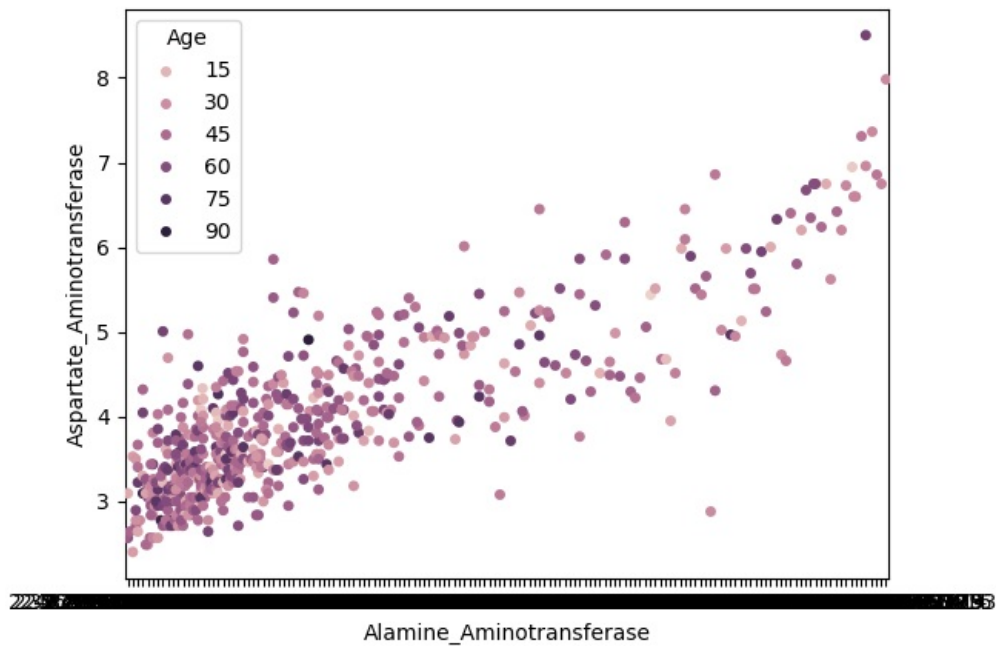
```
Out[55]: <seaborn.axisgrid.PairGrid at 0x1d767e63750>
```



- We clearly see the difference between the before pairplot and after pairplot, the datapoints are equally scattered and normally distributed.

```
In [56]: sns.swarmplot(data = data, x = 'Alamine_Aminotransferase', y = 'Aspartate_Aminotransferase', hue = 'Age')
```

```
Out[56]: <Axes: xlabel='Alamine_Aminotransferase', ylabel='Aspartate_Aminotransferase'>
```



Multicollinearity Problem

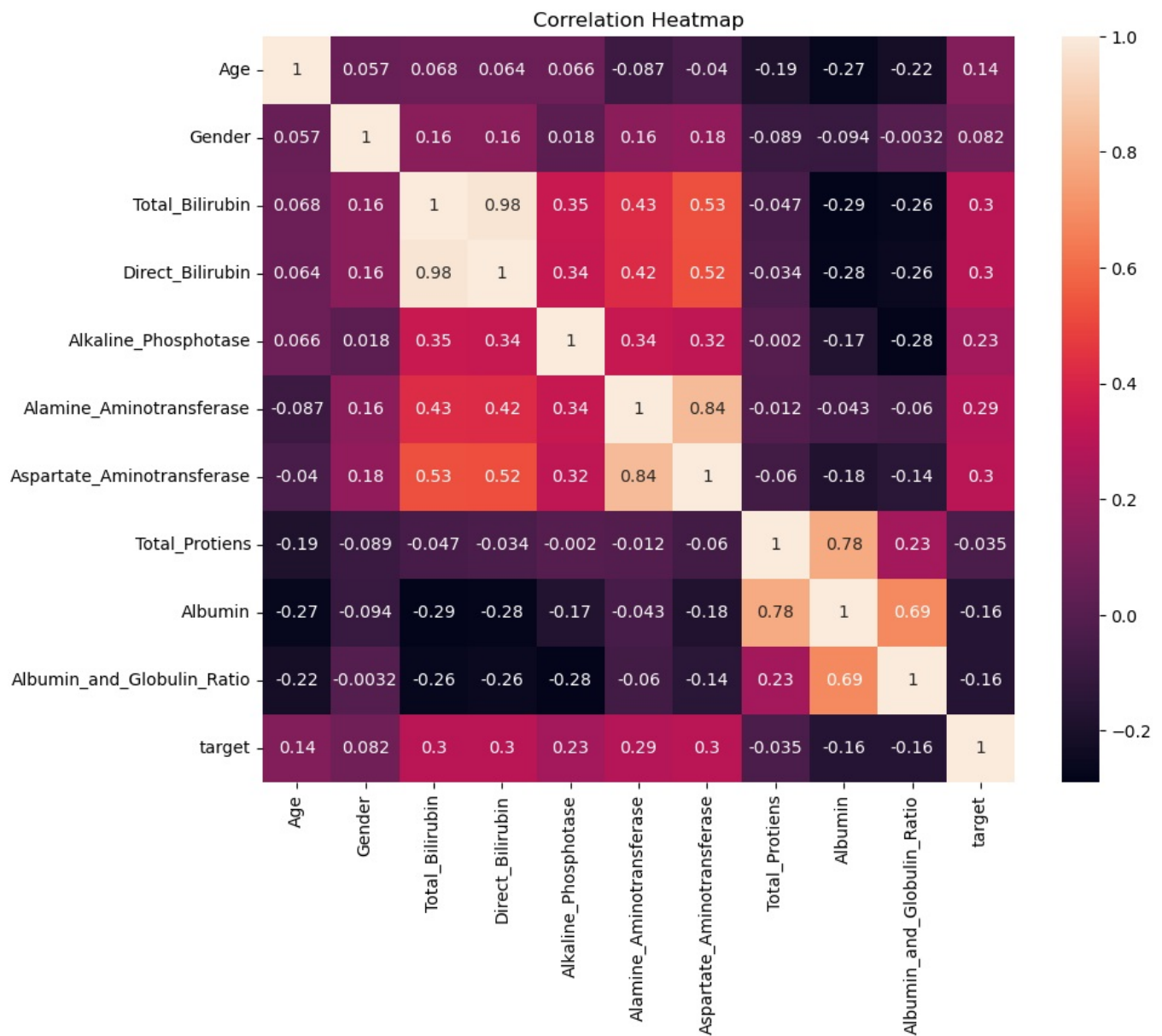
If there are 2 independent features are highly correlated with each other is known as Multicollinearity Problem, which leads to overfitting problem and affects the model stability. There are several ways to check Multicollinearity such as using Heatmap, Variance Inflation Factor(VIF). We'll check the Multicollinearity problem using heatmap.

```
In [57]: data_corr = data.corr()
```

```
In [58]: plt.figure(figsize=(10,8))

sns.heatmap(
    data_corr,
    annot=True,
)

plt.title("Correlation Heatmap")
plt.show()
```



- We can clearly see the Heatmap visulation Total_Bilirubin and Direct_Bilirubin are highly correlated with each other around **98%** this affects the model's performance.
- So we either drop any one of the feature either Total_Bilirubin or Direct_Bilirubin.
- Let's drop Direct_Bilirubin.

```
In [59]: df = data.drop('Direct_Bilirubin', axis=1)
```

```
In [60]: df.head()
```

```
Out[60]:
```

	Age	Gender	Total_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_Aminotransferase	To
0	65	0	0.530628	5.236442	2.833213	2.944439	
1	62	1	2.476538	6.551080	4.174387	4.615121	
2	62	1	2.116256	6.196444	4.110874	4.234107	
3	58	1	0.693147	5.209486	2.708050	3.044522	
4	72	1	1.589235	5.278115	3.332205	4.094345	

```
In [61]: df.shape
```

```
Out[61]: (583, 10)
```

- Since there are no categorical values we dont need Label encoding or one hot encoding.
- The dataset is ready for the model training.

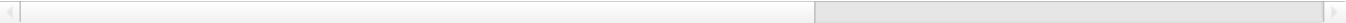
```
In [62]: # Saving the dataset in csv
df.to_csv('Cleaned_Liver_Patient_Dataset.csv', index=False)
```

```
In [63]: # Load the saved dataset
data = pd.read_csv("Cleaned_Liver_Patient_Dataset.csv")
```

```
In [64]: data.head()
```

Out[64]:

	Age	Gender	Total_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_Aminotransferase	To
0	65	0	0.530628	5.236442	2.833213	2.944439	
1	62	1	2.476538	6.551080	4.174387	4.615121	
2	62	1	2.116256	6.196444	4.110874	4.234107	
3	58	1	0.693147	5.209486	2.708050	3.044522	
4	72	1	1.589235	5.278115	3.332205	4.094345	



```
In [65]: data.dtypes
```

Out[65]:

Age	int64
Gender	int64
Total_Bilirubin	float64
Alkaline_Phosphotase	float64
Alamine_Aminotransferase	float64
Aspartate_Aminotransferase	float64
Total_Protiens	float64
Albumin	float64
Albumin_and_Globulin_Ratio	float64
target	int64
dtype:	object